

CS 300 Pseudocode Document

Example Function Signatures

Below is an example of a function signature that you can use as a guide to help address the program requirements using each data structure for the milestones. The pseudocode for finding and printing course information is also given below and depicted in bold to help you get started. The provided pseudocode is for a vector data structure, so you may use this pseudocode in your first milestone as is. The hash table and tree structures are also shown below. But these structures are left for you to do in future milestones.

//Vector - Milestone 1

```
Struct Course
    String courseNumber
    String courseTitle
    Vector<String> prerequisites
End Struct

void loadCourses(Vector<Course> courses, String fileName) {
    open file

    if file does not open
        print error message
        return

    for each line in file
        split line by comma

        if line has less than two items
            print file format error
            continue

        Course course
        course.courseNumber = first item
        course.courseTitle = second item

        for each remaining item
            add item to course prerequisites

        add course to courses
    end for

    close file

    for all courses
        for each prerequisite of the course
            if prerequisite does not exist as a course number
                print prerequisite error
    }
}
```

```
void printCourseList(Vector<Course> courses) {
    sort courses alphanumerically by courseNumber from lowest to
highest

    for all courses
        print course number and course title
}

void searchCourse(Vector<Course> courses, String courseNumber) {
    for all courses
        if the course is the same as courseNumber
            print out the course information

            for each prerequisite of the course
                print the prerequisite course information

            return

    print course not found
}
```

//Hash Table - Milestone 2

```
Struct Course
    String courseNumber
    String courseTitle
    HashTable<String> prerequisites
End Struct

void loadCourses(HashTable<Course> courses, String fileName) {
    open file

    if file does not open
        print error message
        return

    for each line in file
        split line by comma

        if line has less than two items
            print file format error
            continue

        Course course
        course.courseNumber = first item
        course.courseTitle = second item

        for each remaining item
            insert prerequisite into course.prerequisites hash table
```

```
        insert course into courses hash table
        using courseNumber as key
    end for

    close file

    for all courses in hash table
        for all prerequisites in course.prerequisites

            if prerequisite does not exist
                in courses hash table

                print prerequisite error
        }

void printCourseList(HashTable<Course> courses) {

    create empty Vector<Course> sortedCourses

    for all buckets in hash table
        for all courses in bucket
            add course to sortedCourses

    sort sortedCourses alphanumerically by courseNumber from lowest
to highest

    for all courses in sortedCourses
        print course.courseNumber
        print course.courseTitle
}

void searchCourse(HashTable<Course> courses, String courseNumber) {

    if hash table contains courseNumber

        Course course = course found in hash table

        print course.courseNumber
        print course.courseTitle

        if prerequisites hash table is empty
            print "Prerequisites: None"

        else
            print "Prerequisites:"

            for all prerequisites in course.prerequisites

                if courses hash table contains prerequisite

                    Course prereqCourse =
```

```
        prerequisite found in hash table

        print prereqCourse.courseNumber
        print prereqCourse.courseTitle
    else
        print "Course not found"
}
```

//Binary Search Tree - Milestone 3

```
Struct Course
    String courseNumber
    String courseTitle
    Tree<String> prerequisites
End Struct

void loadCourses(Tree<Course> courses, String fileName) {
    open file

    if file does not open
        print error message
        return

    for each line in file
        split line by comma

        if line has less than two items
            print file format error
            continue

        Course course
        course.courseNumber = first item
        course.courseTitle = second item

        for each remaining item
            insert prerequisite into course.prerequisites tree

        insert course into courses tree
        using courseNumber as key
    end for

    close file

    for all courses in tree
        for all prerequisites in course.prerequisites

            if prerequisite does not exist
                in courses tree

                print prerequisite error
```

```
}

void printCourseList(Tree<Course> courses) {

    for all courses in tree in order
        print course.courseNumber
        print course.courseTitle
    }

void searchCourse(Tree<Course> courses, String courseNumber) {

    if tree contains courseNumber

        Course course = course found in tree

        print course.courseNumber
        print course.courseTitle

        if prerequisites tree is empty
            print "Prerequisites: None"

        else
            print "Prerequisites:"

            for all prerequisites in course.prerequisites

                if courses tree contains prerequisite

                    Course prereqCourse =
                        prerequisite found in tree

                    print prereqCourse.courseNumber
                    print prereqCourse.courseTitle
            else
                print "Course not found"
    }
}
```

// Pseudocode for a Menu

Create data structure to store courses

Display menu options:

1. Load Data Structure
2. Print Course List
3. Print Course
9. Exit

Prompt user to enter menu choice

Read userChoice

WHILE userChoice is not 9

```
    IF userChoice equals 1 THEN
        Prompt user for file name
        Read fileName

        Open file
        IF file opens successfully THEN
            Read each line from file
            Parse course information
            Store course in data structure
            Close file
            Display "Courses loaded successfully"
        ELSE
            Display "Error opening file"
        ENDIF
    ELSE IF userChoice equals 2 THEN
        IF data structure is empty THEN
            Display "Please load course data first"
        ELSE
            Sort courses alphanumerically
            FOR each course in sorted list
                Display course number and title
            ENDFOR
        ENDIF
    ELSE IF userChoice equals 3 THEN
        IF data structure is empty THEN
            Display "Please load course data first"
        ELSE
            Prompt user for course number
            Read courseNumber

            Search data structure for matching course

            IF course is found THEN
                Display course number and title
                Display prerequisites

                IF course has prerequisites THEN
                    FOR each prerequisite
                        Display prerequisite
                    ENDFOR
                ELSE
                    Display "No prerequisites"
                ENDIF
            ELSE
                Display "Course not found"
```

```

                                ENDIF
                        ENDIF

                ELSE
                        Display "Invalid menu option"
                ENDIF

                Display menu options again
                Prompt user to enter menu choice
                Read userChoice

        ENDWHILE

        Display "Exiting program"

END

```

Example Runtime Analysis

Vector

The vector is simple and memory efficient, but checking whether prerequisites exist requires searching through the vector, which makes the worst-case runtime slower.

Code	Line Cost	# Times Executes	Total Cost
open file	1	1	1
Check if file opens	1	1	1
For each line in file	1	n	n
Split line by comma	1	n	n
Check for less than two items	1	n	n
Create course object	1	n	n
Assign course number	1	n	n
Assign course title	1	n	n
Add prerequisites	1	n	n
Add course to vector	1	n	n
Close file	1	1	1
Validate prerequisites	n	n	n^2
Total Cost			n^2+8n+3
Runtime			$O(n^2)$

Hash Table

The hash table is very efficient for loading and searching. Its main disadvantage is that it does not naturally store data in sorted order, so printing courses alphabetically requires copying the courses into another structure and sorting them.

Code	Line Cost	# Times Executes	Total Cost
open file	1	1	1
Check if file opens	1	1	1
For each line in file	1	n	n
Split line by comma	1	n	n
Check for less than two items	1	n	n
Create course object	1	n	n
Assign course number	1	n	n
Assign course title	1	n	n
Add prerequisites	1	n	n
Add course to hash table	1	n	n
Close file	1	1	1
Validate prerequisites	1	n	n
Total Cost			9n+3
Runtime			O(n)

Binary Search Tree

The tree keeps courses in sorted order, which is useful for printing the course list alphabetically. However, inserting and searching are slower than a hash table on average.

Code	Line Cost	# Times Executes	Total Cost
open file	1	1	1
Check if file opens	1	1	1
For each line in file	1	n	n
Split line by comma	1	n	n
Check for less than two items	1	n	n
Create course object	1	n	n
Assign course number	1	n	n
Assign course title	1	n	n
Add prerequisites	log n	n	n log n
Add course to vector	log n	n	n log n
Close file	1	1	1
Validate prerequisites	log n	n	n log n
Total Cost			3n log n + 5n + 3
Runtime			O(n log n)

Recommendation

I recommend using a hash table for the ABCU advising program.

The hash table has the best average-case runtime for loading courses and validating prerequisites at $O(n)$. Since advisors will need to quickly search for individual courses, the hash table is the most efficient choice. Although it does not automatically store courses in alphanumeric order, the program can copy the courses into a vector and sort them only when printing the full course list. This makes the hash table the best overall choice for performance and practical use.